

A Microcoded Command Processor for a Flexible FPGA Butterfly Engine

Draft manuscript — author list to be completed
Computer Engineering Laboratory, The University of Sydney

Abstract

FlexBE is a fixed-function FPGA accelerator that executes both long fast Fourier transforms and butterfly-linear neural network layers on a single radix-2 datapath, with the layer sequence fixed at synthesis. An architecture is described that makes the accelerator software programmable while leaving the datapath untouched: a microcoded command processor, implemented as a small sequencer in the programmable logic, executes a queue of 32-byte descriptors that cover data movement, butterfly transforms, and the pointwise operations of the target workload. Two properties of the design are emphasised. The unit of issue is the layer rather than the individual operation, which is appropriate because the datapath retires 64 butterflies per cycle; a single `bs.bfly` descriptor accordingly retires the engine's entire closed-form cycle count. Descriptors name memories and transforms but not bank indices, switch settings, or cycle schedules, so the conflict-free addressing scheme may be revised without invalidating existing programs. The datapath, the architectural state, the descriptor format, and its decode are presented, and a 16-point transform is traced through the machine cycle by cycle. On a model of a Zynq UltraScale+ ZCU104, the sequencer requires 3,640 LUTs (2.4% above the fixed-function system) and completes a 104-descriptor automatic-modulation-classification inference within 8.7% of the hardwired datapath, sustaining 92% of peak throughput at a batch size of ten. The same model indicates that the interactive Python flow used to benchmark the original accelerator is limited by processing-system software rather than by the accelerator, and overstates single-batch latency by a factor of 3.5.

1 Introduction

Edge radio-frequency machine learning (RFML) combines two stages with very different computational profiles. The first is a small number of long transforms; cyclostationary feature extraction, for example, requires four 32,768-point FFTs per classification. The second is a large number of short, highly parallel linear layers. FlexBE [1] accommodates both on one datapath by exploiting the butterfly factorisation of Dao et al. [2], in which an FFT stage and a trained butterfly-linear (BL) layer are the same dataflow evaluated with different 2×2 coefficients. On an AMD ZCU104, the resulting system computes four 32,768-point FFTs in 15,360 cycles (51.2 μ s at 300 MHz) and a complete BSPNet inference in 0.214 ms.

The published system is fixed function. The sequence of

layers, together with their lengths and their interleaving, is embedded in a hardwired top-level state machine, so that any change to the model requires re-synthesis. The question addressed here is what minimal architecture converts the layer graph into runtime data. The answer taken is a microcoded command processor: a descriptor-driven sequencer placed in the programmable logic, following the tradition of microprogrammed control [4] and of the command processors found in contemporary accelerators [11, 9, 10, 8].

Because the datapath retires 64 butterflies per cycle, fine-grained instructions cannot supply work fast enough to be profitable, and on a Zynq MPSoC the hard Cortex-A53 cluster in any case provides no custom-instruction port; both points are quantified in §3. The remaining design choice is the position of the command boundary, and it is argued here that the boundary belongs at the layer. Three consequences of this choice organise the design. First, a single `bs.bfly` descriptor carries the transform length l , the sub-parallelism P_{sub} , and the batch P_N , and retires the engine's closed-form cycle count (Eq. 2); the accelerator's runtime-reconfigurable parameters thus become operand fields rather than configuration registers. Second, no opcode names a RAM bank, a switch setting, or a cycle index, so that the conflict-free addressing of §2 is derived at decode time and remains free to change beneath the instruction set. Third, the pointwise operations are made architectural: magnitude computation alone accounts for 26% of the published system's LUTs, and in the absence of `bs.pow`, `bs.mag`, and `bs.pool` descriptors every feature would be returned to memory and re-read, forfeiting the advantage of the fused datapath.

The contributions of the paper are as follows. The architecture of the command processor is described, including its datapath, architectural state, per-resource descriptor queues, and completion path (§4). The microcode format is specified as a 32-byte descriptor, with its field semantics, encodings, and decode into engine control (§5). A worked example is given at two scales: a 16-point transform traced cycle by cycle through banks, switches, and butterfly units, and a complete 104-descriptor inference with its execution timeline (§6). Finally, an evaluation is carried out on a validated cycle-level model, from which it emerges that the throughput shortfall observed in the published system is attributable chiefly to processing-system software rather than to the accelerator (§7).

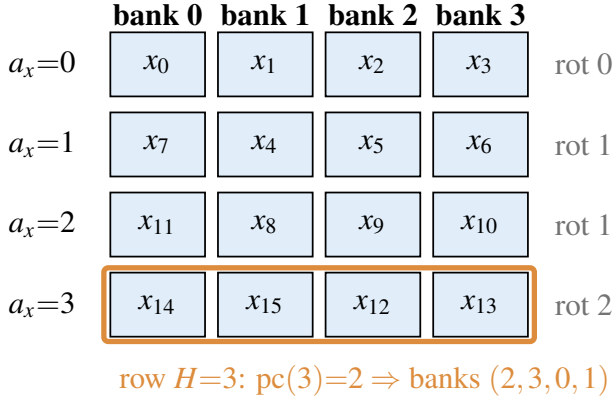


Figure 1: Shift-down storage (Eq. 1) for $N=16$, $P_{bu}=2$ ($P=4$ banks). Row H holds elements $4H..4H+3$ rotated by $pc(H)$ banks, so that any complete row, and, as shown in §2.2, any scheduled cycle, touches every bank exactly once.

2 Background: the Butterfly Engine

The engine driven by the command processor is summarised in this section, in the form implemented by the simulator. Each property stated below is checked by the accompanying test suite.

2.1 Shift-down banked memory

In-flight data is held in $P = 2P_{bu}$ single-port RAM banks. Element index I of an $N = 2^n$ -point working set is stored at depth a_x of bank a_y ,

$$a_x = I \gg m, \quad a_y = \text{bsm}(I) = [\text{pc}(I \gg m) + (I \bmod 2^m)] \bmod 2^m \quad (1)$$

where $m = \log_2 2P_{bu}$ and pc denotes the population count. Equation 1 admits a geometric reading: a RAM row consists of 2^m consecutive indices, and row H is stored cyclically rotated by $pc(H)$ banks (Fig. 1). The conflict-freedom results that follow are consequences of this observation.

2.2 The per-cycle schedule and the PRS

A decimation-in-frequency radix-2 dataflow is used. Stage k of an n -stage transform pairs indices differing in the hole bit $h = n-1-k$, so that natural-order input produces bit-reversed output; the write-back path of §2.4 restores the natural order. In each cycle, $2P_{bu}$ operands are read, permuted into butterfly-slot order, transformed by P_{bu} radix-2 units, inverse-permuted, and written back in place (Fig. 2).

Under Eq. 1, conflict-free access requires two regimes, both of which are produced by the closed-form control $\text{cycle_control}(n, P_{bu}, k, j)$. Writing $\text{ins}(x, p) = ((x \gg p) \ll (p+1)) \mid (x \bmod 2^p)$ for the insertion of a zero bit at position p , the control is

$$\begin{aligned} h < m : H &\leftarrow j; \quad I^i[2t] \leftarrow (H \ll m) + \text{ins}(t, h); \quad S^i \leftarrow h \\ h \geq m : e &\leftarrow j \bmod 2; \quad H \leftarrow \text{ins}(j \gg 1, h-m); \\ I^i[2t] &\leftarrow (H \ll m) + 2t + e; \quad S^i \leftarrow 0 \\ \text{both} : I^i[2t+1] &\leftarrow I^i[2t] + 2^h; \quad R^i \leftarrow \text{bsm}(I^i[0]). \end{aligned}$$

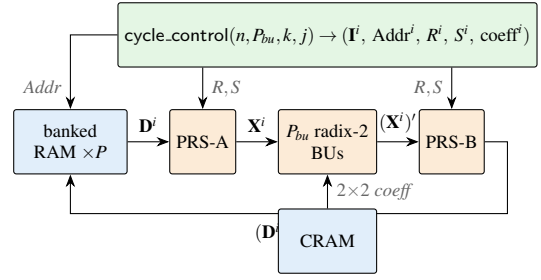


Figure 2: The engine loop. In each cycle one operand is read per bank and gathered into butterfly-slot order (PRS-A), P_{bu} general 2×2 butterflies are applied, and the results are scattered back (PRS-B) and written in place. The coefficient port is private to the engine, so that twiddles and BL weights consume no datapath bandwidth.

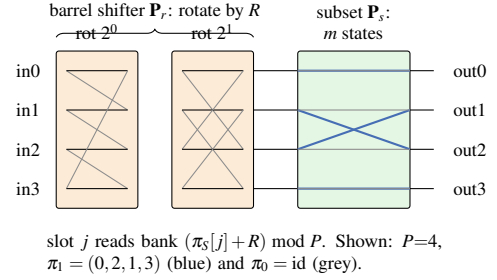


Figure 3: Permute-rotate switch for $P=4$. A rotation network of $\log_2 P$ stages, composed with one of m fixed stride permutations, replaces the all-to-all crossbar at a cost of $\Theta(mP_{bu})$ two-input multiplexers.

When $h < m$, a cycle reads one complete row, whose banks $(pc(H) + low) \bmod 2^m$ are pairwise distinct; slot j then reads bank $(\pi_h[j] + pc(H)) \bmod 2^m$, which is a fixed stride permutation composed with a rotation. When $h \geq m$, the partner element lies in row $H + 2^{h-m}$, whose population count is greater by exactly one, so that slot j reads bank $(pc(H) + e + j) \bmod 2^m$, a pure rotation. The required permutation therefore factors in every case as $\mathbf{P}_f = \mathbf{P}_s \times \mathbf{P}_r$ with at most m subset states, and the switch reduces to a barrel shifter followed by a log-depth subset network (Fig. 3). Its cost is $mP + (m+1)2^{m-1}$ two-input multiplexers per bit, i.e. $\Theta(mP_{bu})$ and 256 at $P_{bu}=16$, compared with the $\Theta(P_{bu}^2)$ of a general crossbar over the same ports (Fig. 11). Each branch emits exactly $N/2P_{bu}$ cycles per stage and visits every index once.

2.3 Sub-parallelism and packing

A datapath of width $2P_{bu}$ would otherwise be obliged to zero-pad any transform shorter than itself. This is avoided by interleaving $P_{sub} = 2P_{bu}/l$ short transforms element-wise into a single computational vector and terminating the recursion after $\log_2 l$ stages; independently, P_N sequences are packed depth-wise into the banked RAM. Both quantities are runtime values and are carried as descriptor fields (§5). The resulting cycle count has the closed form

$$\text{cycles} \approx \frac{P_N N}{2P_{sub} P_{be} P_{bu}} \log_2 \frac{N}{P_{sub}}, \quad (2)$$

in which $N = \max(l, 2P_{bu})$ is the engine computational length and P_N the number of length- l input sequences. At the most

Table 1: cfg-6 single-inference cycle budget on the 4×16 BU array at 300 MHz (from the validated Eq. 2 model).

module	cycles	μ s
CC extraction: 4×32768 -point FFT	15,360	51.20
bit-reversal write-back	1,024	3.41
$8 \times$ feature expansion (len 8, $\times 4$)	24,576	81.92
$8 \times$ MLP-0- $\{0,1\}$ (len 32, 1024 rows)	20,480	68.27
$8 \times$ MLP-1- $\{0,1\}$ (len 32, 128 rows)	2,560	8.53
$8 \times$ MLP-2- $\{0,1\}$ (len 32, 16 rows)	320	1.07
total	64,320	214.4

demanding point of the workload, 4,096 length-8 BL rows per branch on a $P_{bu}=16$ engine, sub-parallelism folds four transforms into each pass and omits two of the five stages, requiring $6.7 \times$ fewer cycles than padding.

2.4 Bit-reversal write-back

Since the dataflow is decimation-in-frequency, the FFT results are produced in bit-reversed order. The write-back path applies $y_k = d_{\text{rev}_m(k)}$ as the engine RAM is streamed to the global buffer. Lane a always reads bank a , the destination bank is $\text{rev}_m((a+u) \bmod P)$, and the permutation completes in exactly N/P cycles without conflict on either memory. In the command set this reordering is expressed as an addressing mode of the store path rather than as a separate instruction (§5); for learned BL layers it is omitted, since the output order is absorbed into the weights of the following layer.

2.5 The workload

BSPNet, used throughout in its cfg-6 configuration, classifies 32,768-sample IQ records into eight modulation classes. The computation comprises the elementwise powers s^2, s^4, s^6, s^8 (Eq. 1 of [1]), four N -point FFTs, and then eight branches, each containing a magnitude operation, a length-8 feature expansion, and three blocks of two length-32 BL layers with a fused LayerNorm–shortcut–ReLU–maxpool operation (“NormPool”). Table 1 reports the per-module cycle budget on the $P_{be} \times P_{bu} = 4 \times 16$ array, totalling 64,320 cycles, or 214.4 μ s at 300 MHz. Two structural properties of this workload shape the architecture: the graph is static, and it contains only about one hundred coarse operations.

3 Why Microcode, and at What Granularity

The datapath retires $P_{be}P_{bu} = 64$ butterflies per cycle. A programmable front end must therefore supply work at a rate of 64 butterflies per issued unit simply to break even, and a control scheme that issued one instruction per butterfly stage per data chunk would require hundreds of butterflies of work per instruction to keep the pipeline occupied. The workload, however, presents only about one hundred coarse operations per 214 μ s inference. The efficient operating point is consequently well defined: the unit of issue should be the layer, and issue should be asynchronous so that data movement is overlapped with computation.

Table 2: Architectural state ($P_{be}=4, P_{bu}=16$).

state	size	visibility
BSPM scratchpad	256 KB	logical index; skew in hardware
CRAM coefficients	64 KB	twiddles / BL weights, Table 4 layout
bsp.cfg	64 b	defaults: l, P_{sub}, P_N , Q-format
bsp.status	64 b	busy mask, queue depth, error, cycles
descriptor rings	3.3 KB	in DDR, fetched over HPC0
tag scoreboard	8 b $\times$ slots	microarchitectural, read via status

The choice of mechanism is further constrained in two respects. The application processors of the target Zynq Ultra-Scale+ MPSoC are hard Cortex-A53 cores that expose no custom-instruction port, so tight coupling in the RoCC style is unavailable and control must be delivered over AXI. Per-command AXI writes are also costly: at approximately 0.4 μ s per posted MMIO write, the 104 commands of an inference amount to some 42 μ s, or 19% of the inference time, before any other overhead is considered, and an interactive Python flow increases this figure by two orders of magnitude (§7). The established response is microprogramming [4] in its modern accelerator form, in which the host writes a program of descriptors to memory once and a small hardware sequencer fetches and executes them [11, 10, 9]. As the command graph is static, the steady-state host cost per inference is reduced to a single pointer update and a single doorbell.

A further principle constrains the content of the descriptors. The conflict-free addressing of §2.2 is a derived property of the memory mapping, and enumerating it in software would fix it in every program. Descriptors are therefore restricted to naming memories, transforms, and shapes, and never a bank, a switch state, or a cycle, so that the schedule remains a microarchitectural matter.

4 Architecture

4.1 System organisation

The machine is shown in Fig. 4. The command processor is positioned between the AXI ports and the fixed-function datapath, and comprises a descriptor fetch and decode unit, three shallow per-resource queues for compute, DMA-in, and DMA-out, a tag scoreboard, and a completion writer. The remainder of the machine is the published engine, used without modification: the BE array, the coefficient RAM, the banked butterfly scratchpad, and the pointwise units.

4.2 Architectural state

The state that a descriptor program may name is listed in Table 2. The butterfly scratchpad (BSPM) is addressed by logical element index, the shift-down skew of Eq. 1 being applied in hardware and hidden from software. This abstraction is central to the design, since it preserves in-place operation and the private coefficient port, both of which are lost in a programmable design based on a register file, while leaving the bank mapping free to change.

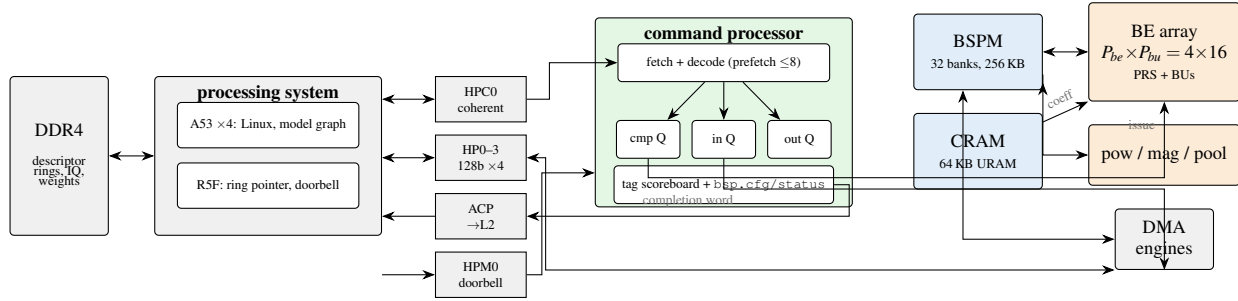


Figure 4: System organisation on the Zynq UltraScale+ MPSoC. The command processor fetches 32-byte descriptors from coherent memory over HPC0, prefetching up to a ring of them, dispatches them into per-resource queues, and retires completions by writing a word through ACP that the host polls in the L2 cache. Bulk IQ data and features are moved over the HP ports, and the doorbell is a single HPM0 write per inference.

4.3 Sequencer microarchitecture

The sequencer operates in three stages. In the fetch stage, descriptors are read over the coherent HPC0 port up to a configurable prefetch depth ahead of retirement, bounded by the ring capacity; a depth of four to eight is shown in §7 to be sufficient. In the decode stage, a descriptor is expanded into engine control. For `bs.bfly` this consists of initialising the loop counters (k, j, seq), whose body is `cycle_control`, together with the CRAM base and the arithmetic mode; the decode reduces to a small number of field extractions. In the dispatch stage, the command is placed in the queue of its resource class. Commands are issued in program order within each resource, and a command begins only once the tags it names have retired. Because dependencies are stated explicitly in the descriptor, the scoreboard is a small table of tag-to-done bits rather than an associative structure. Separating the queues by resource permits the input DMA of the next inference to be fetched and started while the current inference computes; with a single shared queue this descriptor would be held behind the roughly one hundred compute descriptors ahead of it, and §7 quantifies the resulting difference.

Completion is signalled through memory rather than by interrupt. At $214 \mu\text{s}$ per job the Linux interrupt latency of $5\text{--}10 \mu\text{s}$ is significant, so the sequencer writes a completion word through the ACP port, which allocates into the A53 L2 cache, and the host polls it. In the tuned configuration the Cortex-R5F of the real-time unit owns the ring and the doorbell over the low-latency LPD port, leaving the A53 cluster to run Linux.

4.4 Datapath commands

Three units are driven by the compute descriptors. The `bs.bfly` descriptor runs the engine loop of Fig. 2 for $\lceil P_N / P_{\text{sub}} \rceil \cdot (N / 2P_{\text{bu}}) \cdot \log_2(N / P_{\text{sub}})$ cycles per engine, so that Eq. 2 serves directly as its latency formula. The `bs.brev` descriptor runs the write-back network of §2.4 for N/P cycles; the same network serves `bs.store` when the `BITREV` flag is set, so that natural-order output is obtained through an addressing mode rather than an additional pass. The `bs.pow`, `bs.mag`, and `bs.pool` descriptors drive the pointwise units already present in the published system, in which magnitude computation alone occupies 41 k LUTs; treating these units as architectural keeps the inputs of all eight branches on chip.

31	24 23	16 15	8 7	0
log2_l	mode	flags	opcode	
pool	stage_hi	stage_lo	log2_Psub	
src — BSPM / VA source				
dst — BSPM / VA destination				
coeff — CRAM base				
count — elements / bytes				
P_N — sequences of length l				
tag — completion / dependency token				

Figure 5: The 32-byte descriptor (little endian, with word 0 at the top and bit 0 at the right). All shape parameters of the engine, namely l , P_{sub} , P_N , and the stage window, are carried as operand fields.

5 Microcode Format

5.1 Descriptor layout

Each command occupies a single 32-byte word (Fig. 5), consisting of eight single-byte control fields followed by six 32-bit operands. This size corresponds to one beat of a 256-bit AXI read, so that a descriptor is fetched in a single transaction, and the complete `cfg-6` program occupies 3,328 bytes, less than the first few kilobytes of one input record.

5.2 Opcode set and semantics

The eight opcodes, their resource classes, and their latencies in engine cycles or transferred bytes are given in Table 3. Three conventions account for most of the encoding. Shapes are represented as logarithms: the fields `log2_l` and `log2_Psub` render non-power-of-two shapes unrepresentable and decode directly to shift amounts. The stage window [`stage_lo`, `stage_hi`] defaults to the full transform but admits partial passes, which provide both the early termination used by sub-parallelism and a convenient means of debugging. Flags compose: a NormPool block is expressed as a single `bs.pool` descriptor with `LAYERNORM | SHORTCUT | RELU | MAXPOOL`, and BL layers fuse their activation by setting `RELU` on `bs.bfly`.

5.3 Encodings

Five encoded descriptors from the worked example are shown in Table 4, exactly as emitted by the assembler in the accompanying artifact. The second row illustrates the density of the encoding: the entire four-FFT feature-extraction stage, comprising 15,360 datapath cycles, is represented by a single word

Table 3: Opcode set. $C(\cdot)$ abbreviates Eq. 2.

opcode	class	semantics	latency
bs.load	dma-in	mem $\rightarrow$ BSPM	bytes/BW
bs.wload	dma-in	mem $\rightarrow$ CRAM	bytes/BW
bs.store	dma-out	BSPM $\rightarrow$ mem; BITREV, MAG	bytes/BW
bs.bfly	compute	P_N transforms, len l ; FFT/IFFT/ BL/BL_EXPAND; RELU	$C(l, P_{sub}, P_N)$
bs.brev	compute	write-back reversal pass	$P_N N / P$
bs.pow	compute	s^p , $p \in \text{mode}$	$N/128$
bs.mag	compute	$ x $; NORMALISE	$N/128$
bs.pool	compute	LN/shortcut/ReLU/maxpool _f	rows/8

with $\log2_l=15$, $P_N=4$, and $\text{stage_hi}=14$.

5.4 Decode into engine control

The decode of `bs.bfly` is intentionally lightweight. The hardware derives $N = \max(2^{\log2_l}, 2P_{bu})$ and, when $l < 2P_{bu}$, the interleave factor $P_{sub} = 2P_{bu}/l$; it distributes P_N across the P_{be} engines; and it initialises three counters, for the stage k , the cycle j , and the packed sequence. The per-cycle work is then given entirely by the closed form of §2.2: an insert-zero network positioned by the stage-constant hole bit produces $\mathbf{I}^i$, a population count already present in the bsm datapath produces R^i , and S^i is obtained from a comparison of h with m . No field of the descriptor constrains this machinery. The schedule is derived at decode time from the same two-regime rule that the memory mapping imposes, and could be re-derived under a different rule without any change to software.

6 Worked Example

The machine is examined at two scales: a single `bs.bfly` descriptor executing a 16-point FFT, for which every cycle may be tabulated, and the complete `cfg-6` inference, in which the behaviour of interest is the overlap of computation with data movement.

6.1 One descriptor, cycle by cycle

A small engine is configured with $P_{bu} = 2$ ($P = 4$ banks, $m = 2$), and the descriptor

```
bs.bfly{mode=FFT, log2_l=4, P_N=1, stage_hi=3}
```

is issued on input $x_0..x_{15}$ resident in BSPM under the layout of Fig. 1. Decode yields $N=16$, $P_{sub}=1$, and four stages of $N/2P_{bu} = 4$ cycles. The complete control trace emitted by `cycle_control` is given in Table 5, and the data movement of the first cycle is drawn in Fig. 6. Three properties may be verified against the table.

- **Conflict freedom.** Every bsm column is a permutation of $(0, 1, 2, 3)$, without exception across the sixteen cycles. The address column indicates why a per-bank address vector is required in place of a single shared address: in the two-row regime, half of the banks are read at a different depth.
- **Switch economy.** Over all 16 cycles the subset state assumes only the values $\{0, 1\}$, that is h when $h < m$, which

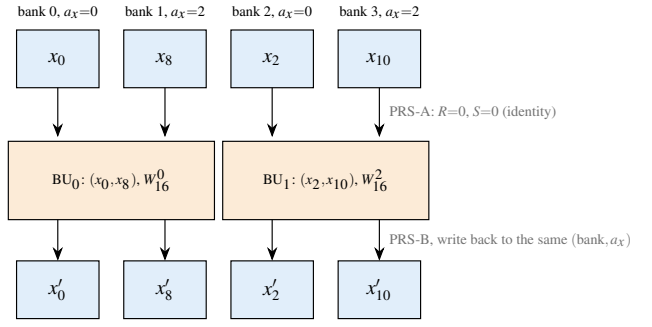


Figure 6: Stage $k=0$, cycle $j=0$ of the worked example. With hole bit $h=3$, the pairs $(0, 8)$ and $(2, 10)$ span rows 0 and 2. The banks $\text{bsm} = (0, 1, 2, 3)$ are distinct, the address vector supplies a different depth per bank, $(0, 2, 0, 2)$, and the permutation is a pure rotation by $R=\text{pc}(0)+e=0$, so that $S=0$. The coefficient addresses $(0, 2)$ select W_{16}^0 and W_{16}^2 .

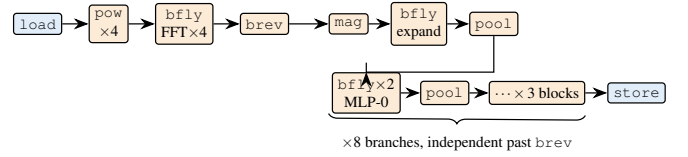


Figure 7: Dependency structure of the `cfg-6` program. The eight branches are independent following the write-back, so that computation does not starve, and the final store joins the tails of all branches.

is two of the $m=2$ available states, while the rotation R^i follows $\text{pc}(H) + e$. This behaviour is what justifies the use of the PRS in place of a crossbar.

- **Latency.** The four stages of 4 cycles give 16 butterfly cycles, in exact agreement with Eq. 2, together with 4 write-back cycles when the result is stored with the BITREV flag. The simulator executes this trace and agrees with `numpy.fft` to within 10^{-15} .

6.2 A complete inference

The head of the `cfg-6` program is listed in Table 6. The program comprises 104 descriptors—1 load, 102 compute, and 1 store, in 3,328 bytes—whose dependency structure is the directed acyclic graph of Fig. 7. The program is constructed by the host once, after which each inference requires only a single pointer update and a single doorbell.

The model’s execution timeline for two consecutive inferences on the tuned platform is shown in Fig. 8. The input DMA of inference 1, shown in orange on the DMA-in lane, is fetched from its own queue and completes entirely within the computation of inference 0; this overlap is precisely what a shared descriptor queue would prevent. The steady-state cost per inference is consequently $240.4 \mu\text{s}$, against a datapath time of $214.4 \mu\text{s}$. The remaining difference is dominated by the 128 KB landing time of the first record, which cannot be overlapped at a batch size of one unless the radio front end is streamed directly into BSPM.

6.3 Host code

The steady-state host code is given in full below.

Table 4: Encoded descriptors (eight little-endian words each; words 0–3 on the first line, 4–7 on the second).

descriptor	encoding (words 0..7)				meaning
bs.load	00000001	001f0000	00000000	00000000	load one 32,768-sample IQ record to BSPM word 0
	00000000	00008000	00000000	00000000	
bs.bfly (FFT)	0f000010	000e0000	00000000	00000000	four 32,768-point FFTs, twiddles at CRAM 0
	00000000	00000000	00000004	00000005	
bs.store (BITREV)	00000103	001f0000	00000000	00000000	bit-reversal write-back fused into the store
	00000000	00008000	00000000	00000006	
bs.bfly (BL)	05020810	00040000	00000000	00000000	1,024 length-32 BL transforms, fused ReLU
	00000040	00000000	00000400	00000009	
bs.bfly (short)	03030010	00020002	00000000	00000000	4,096 length-8 transforms, $P_{sub}=4$ folded per pass
	00000080	00000000	00001000	00000008	

Table 5: Complete control trace of the worked example: 16-point FFT, $P_{bu}=2$. Each row gives, for one cycle, the index vector $\mathbf{I}^i$ in slot order, its banks $\text{bsm}(\mathbf{I}^i)$, which are always a permutation of 0..3, the per-bank address vector Addr^i , the PRS rotation R^i and subset state S^i , and the coefficient addresses of the two butterflies. Stages 0 and 1 are in the two-row regime ($h \geq m$, $S=0$, a pure rotation); stage 2 is in the in-row regime with the stride permutation ($S=1$); and stage 3 is in-row with adjacent pairs ($S=0$).

k	h	j	$\mathbf{I}^i$ (slots 0..3)	$\text{bsm}(\mathbf{I}^i)$	Addr^i (banks 0..3)	R^i	S^i	coeff
0	3	0	(0,8,2,10)	(0,1,2,3)	(0,2,0,2)	0	0	(0,2)
0	3	1	(1,9,3,11)	(1,2,3,0)	(2,0,2,0)	1	0	(1,3)
0	3	2	(4,12,6,14)	(1,2,3,0)	(3,1,3,1)	1	0	(4,6)
0	3	3	(5,13,7,15)	(2,3,0,1)	(1,3,1,3)	2	0	(5,7)
1	2	0	(0,4,2,6)	(0,1,2,3)	(0,1,0,1)	0	0	(0,2)
1	2	1	(1,5,3,7)	(1,2,3,0)	(1,0,1,0)	1	0	(1,3)
1	2	2	(8,12,10,14)	(1,2,3,0)	(3,2,3,2)	1	0	(4,6)
1	2	3	(9,13,11,15)	(2,3,0,1)	(2,3,2,3)	2	0	(5,7)
2	1	0	(0,2,1,3)	(0,2,1,3)	(0,0,0,0)	0	1	(0,1)
2	1	1	(4,6,5,7)	(1,3,2,0)	(1,1,1,1)	1	1	(2,3)
2	1	2	(8,10,9,11)	(1,3,2,0)	(2,2,2,2)	1	1	(4,5)
2	1	3	(12,14,13,15)	(2,0,3,1)	(3,3,3,3)	2	1	(6,7)
3	0	0	(0,1,2,3)	(0,1,2,3)	(0,0,0,0)	0	0	(0,1)
3	0	1	(4,5,6,7)	(1,2,3,0)	(1,1,1,1)	1	0	(2,3)
3	0	2	(8,9,10,11)	(1,2,3,0)	(2,2,2,2)	1	0	(4,5)
3	0	3	(12,13,14,15)	(2,3,0,1)	(3,3,3,3)	2	0	(6,7)

Table 6: Head and tail of the cfg-6 descriptor program.

tag	opcode	operand summary	class	cost	deps
0	bs.load	IQ chunk 0	dma_in	128 KB	–
1	bs.pow	S2	compute	256 cyc	0
2	bs.pow	S4	compute	256 cyc	0
3	bs.pow	S6	compute	256 cyc	0
4	bs.pow	S8	compute	256 cyc	0
5	bs.bfly	4 x 32768-point FFT	compute	15,360 cyc	1,2,3,4
6	bs.brev	write-back pass	compute	1,024 cyc	5
7	bs.mag	branch 0	compute	256 cyc	6
8	bs.bfly	expansion 0	compute	3,072 cyc	7
: (8 branches x 11 descriptors)					
103	bs.store	features	dma_out	2 KB	18..102

```

/* init, once: */
ring = build_cfg6_program(weights); /* 104 x 32 B */
flush(ring);                       /* to coherent DDR */

/* per inference: */
ring[0].src = iq_record;           /* one pointer */
flush(&ring[0]);
mmio_write(DOORBELL, ring_pa);    /* one AXI write */
while (!completion_word);         /* polls in L2 */

```

7 Evaluation

7.1 Methodology

The results are obtained from a cycle-level model validated at three levels. At the engine level, the model executes the dataflow directly, using banked memory with per-cycle conflict checking, the PRS, and general 2×2 butterflies; it agrees with `numpy.fft` to within 10^{-13} across transform sizes, sub-parallel modes, and P_N packing, its measured cycle counts equal Eq. 2, and its Q1.15 arithmetic attains 48.4 dB SQNR at $N=1024$. At the workload level, Eq. 2 is applied to each BSPNet module (Table 1). At the sequencer level, an event-driven model of fetch, queues, DMA, and completion is used, whose only assumptions are the processing-system latencies of Table 7; the timeline of Fig. 8 is produced as its trace output. A suite of 49 tests checks every architectural property claimed in §2–§5, including the encode and decode of descriptors.

7.2 Issue mechanism

The principal result is given in Table 8. Per-descriptor MMIO already falls within 12.0% of the datapath, a static ring within 9.6%, and the tuned configuration within 8.7%. The interactive Python flow used to benchmark the published system, by contrast, is bound by the host: the 104 commands, each incurring an interpreter round trip, yield 878 μs and only 1,139 samples per second. The consequence for the published measurements

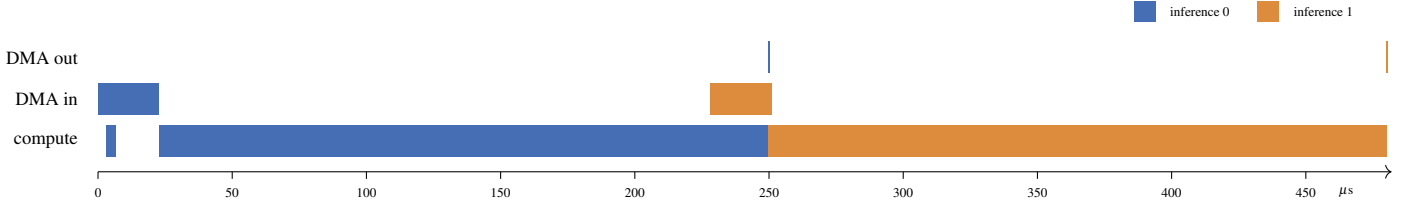


Figure 8: Modelled execution timeline for two inferences on the tuned platform, which uses per-resource queues, streamed input, and an R5-driven ring. The input transfer of inference 1 overlaps the computation of inference 0, and the compute lane runs essentially without gaps. The end-to-end time is 480.7 μ s, or 240.4 μ s per inference.

Table 7: Processing-system timing assumptions, collected in a single dataclass in the artifact.

parameter	value	confidence
posted MMIO write	0.40 μ s	medium
descriptor fetch (HPC0)	0.15 μ s	medium
Python per-transfer cost	25 μ s	soft; conclusion holds 5–50
completion poll via ACP	0.20 μ s	medium
DDR4 sustained	6.4 GB/s	conservative; 10% utilised
sequencer LUTs	3k + 40/slot	rough; 10 $\times$ off is still small

Table 8: cfg-6 single-batch inference by issue mechanism (datapath alone: 214.4 μ s).

mechanism	μ s	ovh. %	samp/s	bound by
PYNQ / Jupyter flow	878.3	281.2	1,139	host
MMIO per descriptor (Linux)	258.1	12.0	3,874	compute
ring rebuilt per inference	252.5	9.6	3,960	compute
static ring + doorbell	252.5	9.6	3,961	compute
static ring, split queues, streamed input	250.3	8.7	3,995	compute

is direct. Most of the gap between the measured and peak throughput curves is attributable to processing-system software and is recoverable with no change to the programmable logic beyond the 3,640-LUT sequencer.

7.3 Queueing behaviour

Prefetch depth is swept in Fig. 9. A depth of four to eight descriptors is sufficient, since the compute commands are long relative to the 0.15 μ s fetch. Batch size is swept in Fig. 10. The shared-ring configuration reaches a plateau early, because the input descriptor of the next inference is held behind the compute descriptors of the current one, whereas per-resource queues allow the DMA to proceed ahead and reach 4,303 samples per second at a batch size of ten, or 92% of the 4,664-samples-per-second datapath peak. This effect is not visible to a bandwidth analysis, as the link is never the binding constraint: 128 KB in and 2.0 KB out per 0.214 ms amount to 0.62 GB/s of the 6.4 available, or 10%, so that a single HP port would suffice and the four ports of Fig. 4 serve only to overlap bursts rather than to raise throughput.

7.4 Cost

The sequencer adds 3,640 LUTs and 2 BRAM36 blocks to the published system, an increase of 2.4% to a total of 156,193 LUTs, against the 15 k LUTs already occupied by the fixed top-

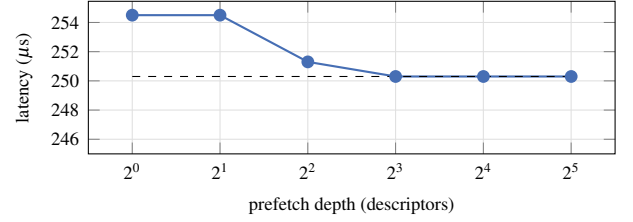


Figure 9: Prefetch-depth sweep at batch size one on the tuned platform. Beyond a depth of four to eight the sequencer is not the limiting factor; the floor, shown dashed, is the computation together with the first input transfer, which cannot be overlapped.

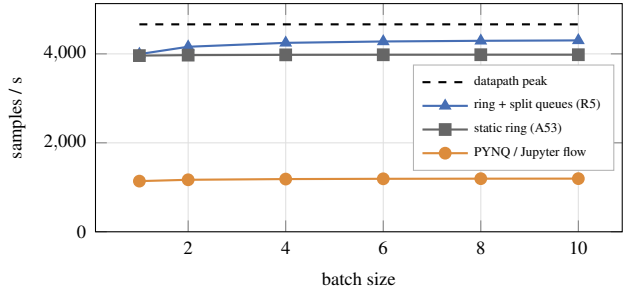


Figure 10: Throughput as a function of batch size. Separate per-resource queues allow the next input transfer to begin during the current computation; the interactive flow does not recover, since every command incurs an interpreter round trip.

level controller. The pointwise descriptors `bs.pow`, `bs.mag`, and `bs.pool` contribute 16.0 μ s of computation, or 7%, which the hardwired system may instead fuse into surrounding operations; where it does, the true overheads are correspondingly lower than those reported here.

8 Related Work

Microprogrammed control originates with Wilkes [4], and the present design is its accelerator-era form, in which the microinstruction is a memory descriptor and the control store a coherent ring. Similar arrangements appear in the NVDLA command interface [11], the task queues of VTA [10], the RoCC command stream of Gemmini [9], and the CISC-style instructions of the TPU [8]. Two features distinguish the design presented here: the command is sized to a layer and has a closed-form latency, and the memory schedule is deliberately excluded from the instruction set. Conflict-free parallel memory access for FFTs derives from Johnson [5] and Takala et al. [6] and is surveyed by Garrido [7]. The butterfly-linear

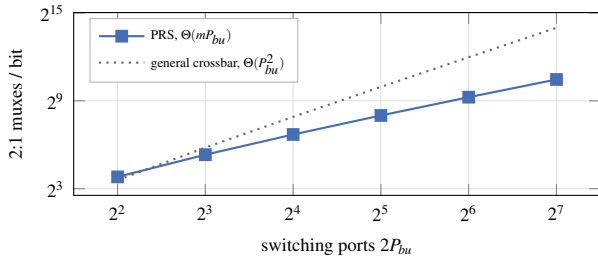


Figure 11: Switching cost of the PRS compared with a general crossbar over the same ports. At 128 ports the figures are 1,408 and 16,256 multiplexers per bit respectively. This economy is preserved under the instruction set precisely because the schedule is kept microarchitectural.

duality that allows one engine to serve both workloads is due to Dao et al. [2] and was first realised in hardware by FABNet [3], of which FlexBE [1], the accelerator programmed here, is the flexible descendant.

9 Limitations and Future Work

The evaluation rests on a validated model rather than on silicon. The sequencer’s timing assumptions (Table 7) remain to be confirmed by board measurement, for which the natural experiment is the measurement of three configurations on a single ZCU104—the hardwired system, the descriptor ring, and the interactive flow—with the published RTL reused unchanged. Two architectural matters are acknowledged rather than resolved. The first is context switching in the presence of a large architectural scratchpad, which is the customary difficulty of this class of design; for an edge appliance the region is made process-private and non-preemptible for the duration of a command, with save and restore through `bs.store` and `bs.load` as the escalation path. The second is that single-batch latency retains an irreducible input landing time of approximately 21 μ s; this could be removed by streaming the software-defined-radio front end directly into BSPM, an ingress path already anticipated by the published system, which is the single most effective improvement available.

10 Conclusion

A fixed-function butterfly accelerator can be made programmable by the addition of a 3,640-LUT sequencer and eight opcodes, provided that the command boundary is placed appropriately: one descriptor per layer, with shapes carried as operands, memories and transforms exposed, and the schedule kept hidden. On a validated model, the resulting machine executes a complete RFML inference within 8.7% of the hardwired datapath and sustains 92% of peak throughput at a batch size of ten. The same model further indicates that the principal performance limitation of the deployed system lay in the processing-system software rather than in the programmable logic.

References

- [1] X. Liu, R. Wu and P. H. W. Leong, “A flexible FPGA-based butterfly engine for accelerating signal processing and machine learning,” manuscript, The University of Sydney.
- [2] T. Dao, A. Gu, M. Eichhorn, A. Rudra and C. Ré, “Learning fast algorithms for linear transforms using butterfly factorizations,” in *ICML*, 2019.
- [3] H. Fan et al., “Adaptable butterfly accelerator for attention-based NNs via hardware and algorithm co-design,” in *MICRO*, 2022.
- [4] M. V. Wilkes and J. B. Stringer, “Micro-programming and the design of the control circuits in an electronic digital computer,” *Proc. Cambridge Phil. Soc.*, 1953.
- [5] L. G. Johnson, “Conflict free memory addressing for dedicated FFT hardware,” *IEEE TCAS-II*, 1992.
- [6] J. Takala, T. Järvinen and H. Sorokin, “Conflict-free parallel memory access scheme for FFT processors,” in *ISCAS*, 2003.
- [7] M. Garrido, “A survey on pipelined FFT hardware architectures,” *J. Signal Processing Systems*, 2022.
- [8] N. P. Jouppi et al., “In-datacenter performance analysis of a tensor processing unit,” in *ISCA*, 2017.
- [9] H. Genc et al., “Gemmini: enabling systematic deep-learning architecture evaluation via full-stack integration,” in *DAC*, 2021.
- [10] T. Moreau et al., “A hardware–software blueprint for flexible deep learning specialization,” *IEEE Micro*, 2019.
- [11] NVIDIA, “NVDLA deep learning accelerator,” open-source hardware documentation, 2017.